Leyes de conservación: teoría, métodos numéricos y aceleración en GPU

Sesión 3: Introducción al High Performance Computing (HPC) y paralelización en GPU parte 2

Dr. Víctor Osores Escalona
Departamento de Matemática, Física y Estadística,
Facultad de Ciencias Básicas,
Universidad Católica del Maule

Universidad Tecnológica de Panamá, 2 al 6 de marzo, 2026

Facultad de Ciencias Básicas

dm2a Doctorado en Modelamiento Matemático Aplicado

Contenido

1. Ejemplos en CUDA parte 2
   - Producto Matriz-Vector
   - Máximo $|x|$ en GPU
2. Métodos numéricos en GPU
3. Implementación
4. Sedimentación polidispersa 1D
5. Referencias

Producto Matriz-Vector: CPU (Python) vs GPU (Numba CUDA)

Objetivo: comparar el cómputo $y = A\,x$ (cada fila de A hace un producto punto con x) en **CPU Python puro** vs **GPU** con un kernel 1D (un hilo por fila).

Estrategia (row-major): 1 hilo → 1 fila

Para cada fila i:

$$y_i = \sum_{j=0}^{n-1} A_{i,j}\, x_j.$$

Paralelismo: filas independientes ⇒ se paraleliza sobre i.

CPU (Python puro)

```
def matvec_py(A, x, y):
    m = A.shape[0]; n = A.shape[1]
    for i in range(m):
        s = 0.0
        for j in range(n):
            s += A[i, j] * x[j]
        y[i] = s
```

GPU (Numba CUDA): 1D grid

```
@cuda.jit
def matvec_rowmajor(A, x, y):
    i = cuda.grid(1)
    m = A.shape[0]; n = A.shape[1]
    if i < m:
        s = 0.0
        for j in range(n):
            s += A[i, j] * x[j]
        y[i] = s
```

Producto Matriz-Vector: CPU (Python) vs GPU (Numba CUDA) `eje3.py`

Lanzamiento del kernel (GPU)

```
threads = 256
blocks  = (m + threads - 1) // threads  # ceil(m/threads)
matvec_rowmajor[blocks, threads](d_A, d_x, d_y)
cuda.synchronize()
```

Interpretación: `i = blockIdx.x * blockDim.x + threadIdx.x`.
Cada hilo calcula exactamente **una** componente $y[i]$ (una fila).

Qué se mide y validación

- **Warm-up GPU:** primera llamada compila el kernel; no se usa para timing.
- **Timing GPU:** mide solo ejecución del kernel (se sincroniza con `cuda.synchronize()`).
- **Timing CPU:** Python puro (doble loop).
- **Speedup:** CPU/GPU; se imprimen primeros valores de y como verificación rápida.

Producto Matriz-Vector: CPU (Python) vs GPU (Numba CUDA)

Lanzamiento del kernel (GPU)

```
threads = 256
blocks  = (m + threads - 1) // threads  # ceil(m/threads)
matvec_rowmajor[blocks, threads](d_A, d_x, d_y)
cuda.synchronize()
```

Nota importante (memoria y tamaño)

Este ejemplo es **muy intensivo en memoria**: A ocupa $m \cdot n$ floats.
Por ejemplo, si $n = 100{,}000$, entonces A tendría 10^{10} entradas $\Rightarrow \approx 40$ GB en `float32`, lo que normalmente **no cabe** en RAM/GPU. En práctica se usa n mucho menor o matrices dispersas / por bloques.

Reducción paralela en GPU: $\max(|x|)$ (ver `max_comp.py`)

Queremos calcular:

$$\max\left(|x[start : start + ncount]|\right)$$

sin copiar el vector completo al CPU.

Estrategia:

- Dividimos el rango en **tiles** de tamaño $2 \cdot \texttt{tpb}$.
- Cada bloque procesa un tile.
- Cada hilo carga hasta **2 elementos**.
- Dentro del bloque se hace una **reducción en árbol**.
- Cada bloque escribe un máximo parcial en `out[bid]`.
- Repetimos el proceso hasta obtener un único valor.

Ventaja: sólo se transfiere **1 escalar** al host.

Ejemplo: $tpb = 8$, $ncount = 25$

Cada bloque procesa:

$$2 \cdot tpb = 16 \text{ elementos}$$

Número de bloques:

$$\left\lceil \frac{25}{16} \right\rceil = 2$$

Para el bloque $bid = 0$:

$$k = bid \cdot (2\,tpb) + tid$$

Con $tid = 0, \ldots, 7$:

$$k = 0, 1, \ldots, 7$$

y además

$$k_2 = k + tpb = 8, \ldots, 15$$

Cada hilo calcula:

$$m = \max\left(|x[start + k]|,\ |x[start + k_2]|\right)$$

Así, el bloque cubre los 16 primeros elementos del rango.

Reducción en árbol (shared memory)

Después de cargar los máximos locales,

$$sm[tid] = m.$$

Se ejecuta la reducción,

$$stride = \frac{tpb}{2}.$$

- stride $= 4$: Hilos `tid=0..3` hacen,

  `sm[0] = max(sm[0], sm[4])`, `sm[1] = max(sm[1], sm[5])`
  `sm[2] = max(sm[2], sm[6])`, `sm[3] = max(sm[3], sm[7])`.

- stride $= 2$: Hilos `tid=0..1` hacen,

  `sm[0] = max(sm[0], sm[2])`, `sm[1] = max(sm[1], sm[3])`.

- stride $= 1$: Hilo `tid=0`, hace,

  `sm[0] = max(sm[0], sm[1])`. (máximo del bloque)

El hilo $tid = 0$ escribe:

$$out[bid] = sm[0]$$

Luego se repite el proceso sobre `out` hasta quedar un único valor.

Preguntas para pensar: ¿$\max(|x|)$, $\min(|x|)$ o suma?

Pregunta 1

La estrategia anterior (tiles + shared memory + reducción en árbol) sirve también para calcular:

$$\min\left(|x[start : start + ncount]|\right) \quad \text{o} \quad \sum_{k=0}^{ncount-1} x[start + k] \ ?$$

Pregunta 2: ¿Cómo lo harían? (pistas)

- **Para** $\min(|x|)$**:** ¿qué cambio harías en el operador de reducción? ¿con qué valor inicial (identidad) deben partir los hilos fuera de rango?
- **Para la suma:** ¿qué cambio harías en la reducción en shared memory? ¿qué problemas de **precisión** aparecen al sumar en paralelo (`float32`)?

Idea clave: mantener el mismo patrón de paralelismo (tiles + reducción), pero cambiar la **operación** (`max` → `min` o +) y el **valor neutro**.

¿Y cómo calcular $\max(x)$ y $\min(x)$?

Hasta ahora hemos calculado

$$\max(|x|)$$

porque esta cantidad aparece frecuentemente en los **métodos numéricos** estudiados (por ejemplo en estimaciones de estabilidad o criterios tipo CFL).

Pero muchas aplicaciones requieren también calcular: $\max(x)$, $\min(x)$.

Observación:
La estrategia paralela es exactamente la misma:

- dividir el vector en **tiles**
- cada bloque procesa un tile
- usar **shared memory**
- aplicar una **reducción en árbol**

La única diferencia es el **operador de reducción**: $\max(a, b)$ o $\min(a, b)$ en lugar de $\max(|a|, |b|)$.
Es decir, basta reemplazar $|x[i]| \to x[i]$ y usar el operador correspondiente.
Pregunta: ¿qué elección debemos hacer para las **semillas iniciales**?

Métodos de Volúmenes Finitos (FV) para leyes de conservación

Consideremos una ley de conservación 1D:

$$u_t + f(u)_x = 0, \qquad u(x,t) \in \mathbb{R}.$$

Discretización FV (celdas + flujos en interfaces)

Sea $u_i^n \approx \frac{1}{\Delta x}\int_{x_{i-1/2}}^{x_{i+1/2}} u(x,t^n)\,dx$. El esquema FV explícito se escribe como

$$u_i^{n+1} = u_i^n - \lambda\Big(F_{i+1/2}^n - F_{i-1/2}^n\Big), \qquad \lambda = \frac{\Delta t}{\Delta x},$$

donde $F_{i+1/2}^n = \mathcal{F}(u_i^n, u_{i+1}^n)$ es un **flujo numérico** que aproxima el flujo físico.

Estabilidad (criterio CFL)

Para Burgers $f(u) = \frac{1}{2}u^2$ y velocidad característica $a(u) = f'(u) = u$, típicamente se impone

$$\Delta t \le \mathrm{CFL}\,\frac{\Delta x}{\max_i |a(u_i^n)|} = \mathrm{CFL}\,\frac{\Delta x}{\max_i |u_i^n|}.$$

En GPU, $\max_i |u_i^n|$ se calcula con una **reducción paralela**.

Flujos numéricos: Rusanov, Roe, HLL y Lax-Friedrichs

En cada interfaz usamos estados izquierdo/derecho:

$$(u_L, u_R) = (u_i^n, u_{i+1}^n), \qquad f_L = f(u_L),\ f_R = f(u_R).$$

Para Burgers: $f(u) = \frac{1}{2}u^2$, $a(u) = u$.

Familia tipo central + disipación

Muchos flujos se escriben como

$$\mathcal{F}(u_L, u_R) = \frac{1}{2}(f_L + f_R) \ - \ \frac{1}{2}\,\alpha(u_L, u_R)\,(u_R - u_L),$$

donde $\alpha \geq 0$ controla la **viscosidad numérica** (estabilidad vs difusión).

Flujos numéricos: Rusanov, Roe, HLL y Lax-Friedrichs (`FV_cuda.py`)

(1) Rusanov / LF local

$$\alpha = \max(|u_L|, |u_R|).$$

Pros: robusto, simple.
Contras: más difusivo.

(2) Roe (escalar)

$$\alpha = \left|\tilde{a}\right|, \qquad \tilde{a} = \frac{u_L + u_R}{2}.$$

Pros: menos difusivo.
Nota: puede requerir entropy fix en choques.

(3) HLL

Sea $S_L = \min(u_L, u_R)$, $S_R = \max(u_L, u_R)$. Entonces

$$\mathcal{F} = \begin{cases} f_L, & 0 \le S_L, \\ f_R, & 0 \ge S_R, \\ \dfrac{S_R f_L - S_L f_R + S_L S_R (u_R - u_L)}{S_R - S_L}, & S_L < 0 < S_R. \end{cases}$$

Pros: muy robusto, buen compromiso.

(4) Lax-Friedrichs global

$$\mathcal{F} = \frac{1}{2}(f_L + f_R) - \frac{1}{2}\frac{\Delta x}{\Delta t}(u_R - u_L).$$

Pros: muy estable. **Contras:** muy difusivo.

Tarea: Modelos escalares 1D y Volúmenes Finitos en GPU

Contexto

Trabajaremos con leyes de conservación escalares en 1D:

$$u_t + (f(u))_x = 0,$$

usando un esquema de **volúmenes finitos** con **flujos numéricos** (Rusanov / Roe / HLL / Lax-F). La meta es implementar y comparar variantes en **Numba CUDA**.

Objetivos de aprendizaje

- Implementar FV 1D con **ghost cells** y condiciones de borde.
- Implementar flujos numéricos: **Rusanov**, **Roe** (escalar), **HLL** y **Lax-F** global.
- Calcular $\max |f'(u)|$ para CFL usando **reducción en GPU**.
- Validar con un benchmark (advección lineal) y explorar fenómenos (choques/rarefacciones).

Restricción

Debe reutilizarse la estructura base entregada (`ghosts + max_abs_device + FV step`).

Modelos a implementar

(A) Tráfico 1D: LWR (Greenshields)

$$\rho_t + (f(\rho))_x = 0, \qquad f(\rho) = \rho(1-\rho), \qquad 0 \le \rho \le 1.$$
$$f'(\rho) = 1 - 2\rho, \qquad \max |f'(\rho)| \le 1.$$

(B) Buckley-Leverett (flujo bifásico en porosos)

$$s_t + (f(s))_x = 0, \qquad f(s) = \frac{s^2}{s^2 + M(1-s)^2}, \qquad 0 \le s \le 1.$$

Nota: f es no lineal y puede cambiar convexidad $\Rightarrow$ choques/rarefacciones clásicos.

(C) Advección lineal (benchmark)

$$u_t + a\, u_x = 0, \qquad f(u) = au, \qquad f'(u) = a, \qquad u(x,t) = u_0(x - at).$$

Esquema FV 1D + CFL (lo que deben programar)

Update FV (malla uniforme)

$$u_i^{n+1} = u_i^n - \frac{\Delta t}{\Delta x}\left(F_{i+\frac{1}{2}} - F_{i-\frac{1}{2}}\right).$$

CFL (estabilidad)

$$\Delta t \leq \mathrm{CFL}\,\frac{\Delta x}{\max_i |f'(u_i)|}.$$

Requisito: estimar $\max|f'|$ con **reducción en GPU** (misma idea que $\max(|x|)$).

Condiciones de borde (con ghosts, $ng = 1$)

- Outflow: $u_0 = u_1, \quad u_{n+1} = u_n.$
- (Opcional) Periodic: $u_0 = u_n, \quad u_{n+1} = u_1.$

Entregables (lo que deben entregar)

1) Código

- Un archivo `fv_gpu.py` con:
  - `f(u)` y `fp(u)` para cada modelo.
  - `flux_num(uL,uR,method)` con 4 métodos.
  - Kernel FV (update) con ghosts.
  - Reducción GPU para $\max |f'(u)|$ (CFL).
- Scripts de prueba: `test_advection.py` y `test_nonlinear.py`.

2) Reporte corto (máx. 2 páginas)

- Explicar qué modelo implementaron y por qué es relevante (aplicación).
- Mostrar 2 figuras: solución inicial vs final, y comparación de métodos.
- Reportar tiempos CPU vs GPU (aunque sea aproximado).

Checklist de validación (antes de comparar métodos)

Validación mínima (obligatoria)

- **Advección lineal:** comparar con solución exacta $u(x,t) = u_0(x - at)$.
- Verificar que el CFL usado coincide con $|a|$ (constante).
- Revisar que el método conserva **masa** (aprox.):

$$\sum_i u_i^n \Delta x \approx \text{cte.}$$

Errores típicos (si aparece "algo raro")

- Ghosts mal actualizados (BC en u pero no en u^{n+1}).
- Índices de interfaces ($F_{i+1/2}$) corridos en 1.
- CFL calculado con el vector equivocado (incluyendo ghosts o no).

Modelo de sedimentación polidispersa

Consideramos N especies sólidas dispersas en un fluido viscoso en 1D (altura z).

$$\frac{\partial \phi_i}{\partial t} + \frac{\partial}{\partial z}\Big(\phi_i\, v_i^{\mathrm{MLB}}(\Phi)\Big) = 0, \qquad i = 1, \ldots, N,$$

con condición inicial y condiciones de borde apropiadas. Aquí $\Phi = (\phi_1, \ldots, \phi_N)$ y el conjunto admisible es

$$\boldsymbol{\mathcal{D}} = \Big\{\Phi \in \mathbb{R}^N : \ \phi_i \geq 0, \ i = 1, \ldots, N; \ \ \phi_1 + \cdots + \phi_N \leq \phi_{\max}\Big\}.$$

La velocidad de sedimentación obstaculizada (Masliyah-Lockett-Bassoon) es

$$v_i^{\mathrm{MLB}}(\Phi) = \mu\, V(\phi) \left[\delta_i \left(\bar{\rho}_i - \bar{\rho}^{\mathsf{T}}\Phi\right) - \sum_{l=1}^{N} \delta_l\, \phi_l \left(\bar{\rho}_l - \bar{\rho}^{\mathsf{T}}\Phi\right)\right], \quad i = 1, \ldots, N,$$

donde $\phi = \sum_{i=1}^{N} \phi_i$, $\bar{\rho}_i := \rho_i - \rho_0$, $\bar{\rho} := (\bar{\rho}_1, \ldots, \bar{\rho}_N)^{\mathsf{T}}$,

$$\mu := -\frac{g\, d_1^2}{18\, \mu_{\mathrm{f}}}, \qquad \delta_i := \frac{d_i^2}{d_1^2},$$

y el factor de obstaculización es

$$V(\phi) = \begin{cases} (1-\phi)^{\lambda-2}, & \Phi \in \boldsymbol{\mathcal{D}},\ \lambda > 2, \\ 0, & \text{en otro caso.} \end{cases}$$

Método numérico (FVM) para el modelo polidisperso (`sedpoly1d_cuda.py`)

Actualización FVM,

$$\phi_{i,j}^{n+1} = \phi_{i,j}^{n} - \frac{\Delta t_n}{\Delta z}\left(\hat{f}_{i,j+\frac{1}{2}}^{n} - \hat{f}_{i,j-\frac{1}{2}}^{n}\right), \qquad i = 1, \ldots, N.$$

Flujo numérico,

$$\begin{aligned}\hat{f}_{i,j+\frac{1}{2}}^{n} =& \frac{1}{2}\left(\phi_{i,j+1}^{n} v_i(\Phi_{j+1}^{n}) + \phi_{i,j}^{n} v_i(\Phi_{j}^{n})\right) - \frac{|v_i(\Phi_{j+1}^{n})|}{2}\left(\phi_{i,j+1}^{n} - \phi_{i,j}^{n}\right) \\ &- \frac{\phi_{i,j}^{n}}{2}\left|v_i(\Phi_{j}^{n}) - v_i(\Phi_{j+1}^{n})\right| \operatorname{sgn}\left(\phi_{i,j+1}^{n} - \phi_{i,j}^{n}\right).\end{aligned}$$

Paso de tiempo (CFL),

$$\Delta t_n = 0.5\,\frac{\Delta z}{\max\limits_{j=0,\ldots,\mathcal{N}-1}\ \max\limits_{i=1,\ldots,N}\left|v_i(\Phi_j^n)\right|}.$$

+ C. I y C. F en $z_{-\frac{1}{2}} = 0$ y $z_{\mathcal{N}-\frac{1}{2}} = L$.

Nota: el flujo combina un término centrado con disipación tipo upwind y un corrector no lineal para estabilizar gradientes.

References

- S. Böhm. How to Optimize a CUDA Matrix Multiplication Kernel. Personal technical blog, 2022. Available at: https://siboehm.com/articles/22/CUDA-MMM
- NVIDIA Corporation. cuBLAS Library User Guide. NVIDIA CUDA Toolkit Documentation. Available at: https://docs.nvidia.com/cuda/cublas/index.html
- M. Harris. Optimizing Parallel Reduction in CUDA. NVIDIA Developer Technology, 2007. Available at: https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf
- V. Volkov and J. Demmel. Benchmarking GPUs to tune dense linear algebra. Proceedings of SC08: International Conference for High Performance Computing, Networking, Storage and Analysis, 2008.
- NVIDIA Corporation. CUDA C Programming Guide. NVIDIA CUDA Toolkit Documentation. Available at: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

Agradecimientos

Gracias por su atención

Quedo atento a sus preguntas y comentarios.